

Data Integrity Bug Report

Sprint review — April 19, 2026 | Severity: Critical & Medium

Bug 1 — Mixed-Case Email Breaks Bulk-Uploaded Account Login

CRITICAL

Affected files: addStudentsController.js : 210 | addStaffBatchController.js : 218
Discovered: April 19, 2026 (code audit)

User Story

As an administrator uploading a batch of student or staff accounts, I want every uploaded email to be stored in lowercase, so that newly created users can log in immediately without encountering an "Invalid credentials" error caused by email case mismatch.

Problem Description

When an admin uploads a CSV or Excel file containing a mixed-case email such as "Student@College.edu", both bulk-upload controllers store that exact value in the users table without normalising the case.

Every authentication path in authController.js lowercases the incoming email before running `SELECT ... WHERE email = $1`. Because the stored value is "Student@College.edu" but the query uses "student@college.edu", PostgreSQL's case-sensitive equality check finds no match and returns "Invalid credentials" — permanently locking the user out of their own account.

Root Cause

The duplicate-check query (line 198 / 204) correctly uses `LOWER($1)`, so duplicates are detected. However, the `INSERT` that follows uses the raw `s.email` / `staff.email` value taken directly from the spreadsheet row. The normalisation step present in the self-registration flow (authController.js : 59 — `const emailLower = email.toLowerCase()`) is absent from both batch controllers.

Affected Code

```
// addStudentsController.js : 210 — BUG: s.email not lowercased
await tracedQuery(client,
  `INSERT INTO users (email, ...) VALUES ($1, ...)\`,
  [s.email, hash, ...] // <-- stores mixed-case
);

// addStaffBatchController.js : 218 — same issue
await tracedQuery(client,
  `INSERT INTO users (email, ...) VALUES ($1, ...)\`,
  [staff.email, hash, ...] // <-- stores mixed-case
);
```

Proposed Fix

```
// addStudentsController.js
const emailNorm = s.email.toLowerCase();
await tracedQuery(client,
  `INSERT INTO users (email, ...) VALUES ($1, ...)\`,
  [emailNorm, hash, ...]
);

// addStaffBatchController.js
const emailNorm = staff.email.toLowerCase();
await tracedQuery(client,
```

Acceptance Criteria

- + A CSV row with email "Student@College.edu" creates a user stored as "student@college.edu".
- + That user can log in immediately using either "student@college.edu" or "Student@College.edu".
- + Duplicate detection still works correctly after the fix.
- + Existing (already stored) mixed-case emails are unaffected by this change.
- No regression in the self-registration flow (which already lowercases correctly).

MEDIUM

Affected file: dataUploadController.js : 220 – 252 (saveUploadedData loop)

Discovered: April 19, 2026 (code audit)

User Story

As a staff member uploading a large batch of achievements or projects, I want the error report to list only the rows that actually contain invalid data, so that I can correct exactly those rows and re-upload without wasting time investigating hundreds of rows that were never broken.

Problem Description

The batch-insert loop in `saveUploadedData` wraps all rows inside a single `BEGIN ... COMMIT` block and catches per-row exceptions individually. When a row throws any error other than a duplicate-key violation (23505), PostgreSQL immediately puts the entire transaction into an "aborted" state.

Every subsequent query in the same transaction — including perfectly valid rows — is rejected by PostgreSQL with: `ERROR: current transaction is aborted, commands ignored until end of transaction block`

Each of those rejections is caught by the per-row handler and added to `errors[]`. The final response reports all remaining rows as failures. In a 500-row upload where only row 5 is broken, the admin sees 496 phantom errors and cannot identify the real problem.

Root Cause

No `SAVEPOINT` is established before each row's insert attempt. Without a savepoint, a statement-level error escalates to a transaction-level abort in PostgreSQL, making all further statements in that transaction impossible. The `ROLLBACK` at line 255 is correct and prevents data corruption, but the error report produced before the rollback is misleading.

Affected Code

```
for (let i = 0; i < rows.length; i++) {
  try {
    // No SAVEPOINT – a failure here aborts the whole transaction
    await saveAchievement(client, normalized, user, i + 2);
    created++;
  } catch (err) {
    if (err.code === '23505') {
      skipped.push({ row: i + 2, reason: 'Duplicate entry' });
    } else {
      // All rows after the first error also land here (aborted state)
      errors.push({ row: i + 2, error: err.message });
    }
  }
}
```

Proposed Fix

Wrap each row in a `SAVEPOINT` so failures are row-scoped, not transaction-scoped:

```
for (let i = 0; i < rows.length; i++) {
  const sp = `row_${i}`;
  await client.query(`SAVEPOINT ${sp}`);
  try {
    if (dataType === 'achievements')
      await saveAchievement(client, normalized, user, i + 2);
    // ... other types ...
    created++;
  } catch (err) {
    await client.query(`ROLLBACK TO SAVEPOINT ${sp}`);
```

With this fix the transaction remains active after a row-level failure. Valid rows continue to insert successfully, duplicates are correctly skipped, and the error report reflects only genuinely broken rows. The COMMIT / ROLLBACK logic at lines 254–265 is unchanged.

Acceptance Criteria

- + Uploading 500 rows where row 5 has an invalid value reports exactly 1 error (row 5).
- + The remaining 499 valid rows are inserted and committed successfully.
- + Duplicate rows are still categorised as "skipped", not "errors", even after a failed row.
- + If all rows are invalid, the response reports all rows as errors (no partial commit).
- No regression in the all-or-nothing rollback when intentionally triggered.
- + Existing tests continue to pass.

Developer		
QA Engineer		
Product Owner		

